

Distribution and Time Series Charts in R: Histograms to Forecasts

Radar Chart – Theory

A radar chart (also called spider or web chart) maps multiple numeric variables onto radial axes emanating from a common center. Each axis represents one variable, and a polygon is drawn connecting the values — making it easy to compare profiles across observations or groups.

Structure

Each axis is scaled independently from 0 to its maximum. The polygon area reflects the overall "magnitude" of the profile.

Best use cases

Comparing a small number of entities (≤ 5) across the same set of variables; e.g., species profiles, model performance metrics.

Limitations

Perception of area can be misleading; avoid when variables have very different scales or when there are more than ~8 axes.

- ❏ Use radar charts when you need to compare multivariate profiles at a glance. Avoid them for precise quantitative comparisons — a grouped bar chart is more accurate for that purpose.

Ref.: Wilkinson, L. (2005). *The Grammar of Graphics* (2nd ed.). Springer. — Chapter on multivariate displays.

Radar Chart

Building on our exploration of relationships between variables, let's examine one powerful way to visualize **multiple numeric variables simultaneously**. The radar chart displays multiple numeric variables on radial axes from a common origin — useful for **comparative profiles**. Data must be formatted with name and value before plotting.

```
data <- iris |> dplyr::select(-Species)
data <- sapply(data, mean)
data <- data.frame(name=names(data), value=data) |>
  dplyr::arrange(name)

grf <- plot_radar(data, colors=colors[1]) + font
grf <- grf + ylim(0, NA)
```

Now that we've seen how radar charts reveal comparative profiles across variables, let's explore how individual variables are distributed — starting with the histogram.

📄 Ref.: Wilkinson, L. (2005). *The Grammar of Graphics*. Springer.

Histogram – Theory

A histogram partitions a continuous variable into contiguous intervals (bins) and counts how many observations fall in each bin. It is the most direct way to visualize the empirical distribution of a single variable.

How it works

The X-axis shows the value range divided into bins of equal width. The Y-axis shows frequency (count) or density. Bar height encodes how many observations fall in each interval.

What it reveals

Shape (symmetric, skewed left/right), modality (unimodal, bimodal), spread, and the presence of outliers or gaps in the data.

Key parameter: bin width

Too few bins hide structure; too many bins create noise. A good rule of thumb is Sturges' rule: $k = \lceil \log_2(n) \rceil + 1$, or use Scott's or Freedman–Diaconis rules for larger datasets.

- ❏ The histogram is the starting point for any distributional analysis. Always inspect it before choosing a statistical model — the shape of the distribution directly informs which assumptions are valid.

Ref.: Scott, D. W. (1992). *Multivariate Density Estimation: Theory, Practice, and Visualization*. Wiley. — Foundational reference on bin-width selection and density estimation.

Histogram

Building on the radar chart's comparative profiles, we now examine how individual variables are distributed. The histogram reveals the shape, skewness, and outliers of continuous data.

```
library(daltoolbox)
library(ggplot2)
library(RColorBrewer)
library(dplyr)
library(gridExtra)

colors <- brewer.pal(4, 'Set1')
font <- theme(text = element_text(size=16))

# Generate variables with distinct distributions
example <- data.frame(exponential = rexp(100000, rate = 1),
  uniform = runif(100000, min=2.5, max=3.5), normal = rnorm(100000, mean=5))

# Simple histogram
grf <- plot_hist(example |> dplyr::select(exponential),
  label_x = "exponential", color = colors[1]) + font
plot(grf)
```

```
# Multiple histograms side by side
grfe <- plot_hist(example |> select(exponential), label_x="exponential", color=colors[1]) +
font
grfu <- plot_hist(example |> select(uniform), label_x="uniform", color=colors[1]) + font
grfn <- plot_hist(example |> select(normal), label_x="normal", color=colors[1]) + font

grid.arrange(grfe, grfu, grfn, ncol=3)
```

Step by step

- Generate data with `rexp()`, `runif()` and `rnorm()` to simulate distinct distributions
- Call `plot_hist()` passing the selected column and the X-axis label
- For multiple plots, create each one separately and combine with `grid.arrange(..., ncol=3)`

📌 **Key concept:** The histogram distributes observations into *bins* on the X-axis and counts the frequency in each bin. It reveals the shape, skewness, and outliers of continuous variables.

Now let's explore a smoother representation of these distributions — the density plot offers a continuous curve that refines what the histogram approximates with discrete bins.

Density Estimation – Theory

Kernel Density Estimation (KDE) is a non-parametric method for estimating the probability density function of a continuous variable. Unlike the histogram, it produces a smooth, continuous curve that does not depend on arbitrary bin boundaries.

How it works

A kernel function (typically Gaussian) is centered at each data point. These individual "bumps" are summed and normalized to produce a smooth density curve. The result integrates to 1, making it a proper probability density.

Bandwidth (h)

The key tuning parameter. A small bandwidth captures fine detail but may overfit (noisy curve); a large bandwidth over-smooths and hides structure. Common selectors: Silverman's rule of thumb, cross-validation.

Advantages over histograms

No bin-boundary artifacts; continuous and differentiable; easier to overlay multiple distributions for comparison; better for detecting multimodality.

- ❏ KDE is especially useful when comparing distributions across groups — overlaying multiple density curves on the same plot is far cleaner than overlaying multiple histograms.

Ref.: Silverman, B. W. (1986). *Density Estimation for Statistics and Data Analysis*. Chapman & Hall. — The definitive reference on KDE theory and bandwidth selection.

Density (Kernel Density)

Building on the histogram's bin-based view, we now examine a *smoothed* alternative that reveals distribution shape without depending on bin size.

```
library(daltoolbox)

colors <- brewer.pal(4, 'Set1')
font <- theme(text = element_text(size=16))

# Data with distinct distributions
example <- data.frame(exponential = rexp(100000, rate = 1),
  uniform = runif(100000, min=2.5, max=3.5),
  normal = rnorm(100000, mean=5))

# Combined density (3 variables)
grf <- plot_density(example, colors=colors[1:3]) + font
plot(grf)

# Individual densities side by side
grfe <- plot_density(example |> select(exponential),
  label_x="exponential", color=colors[1]) + font
```

```
grfu <- plot_density(example |> select(uniform),
  label_x="uniform", color=colors[2]) + font
grfn <- plot_density(example |> select(normal),
  label_x="normal", color=colors[3]) + font

grid.arrange(grfe, grfu, grfn, ncol=3)
```

Step by step

- Use the same data as the histogram to compare both approaches
- `plot_density(example, colors=colors[1:3])` overlays the three curves in a single chart
- For individual plots, select each column with `dplyr::select()` and combine with `grid.arrange()`
- Each color should correspond to a distinct variable

Next, we examine the **boxplot** — a compact summary that highlights median, spread, and outliers across groups.

📌 **Key concept:** Kernel density estimation is a *smoothed* version of the histogram for continuous data. It uses `geom_density()` internally and highlights the shape of the distribution without depending on bin size.

Boxplot – Theory

The boxplot (or box-and-whisker plot) is a compact, quartile-based summary of a distribution. Introduced by John Tukey in 1977, it encodes five key statistics in a single graphic and excels at comparing distributions across multiple groups.

The five-number summary

Minimum, Q1 (25th percentile), Median (Q2), Q3 (75th percentile), Maximum. The box spans the interquartile range ($IQR = Q3 - Q1$), with a line at the median.

Whiskers

Extend to the most extreme values within $1.5 \times IQR$ from the box edges. Data points beyond the whiskers are plotted individually as potential outliers.

Group comparison

Placing multiple boxplots side by side immediately reveals differences in location (median), spread (IQR), and skewness across groups.

Limitations

Hides multimodality and sample size. For small datasets, consider overlaying individual points (strip chart or jitter) on top of the boxplot.

- ❏ A boxplot does not show the full shape of the distribution — two very different distributions can produce identical boxplots. Combine with a density plot or strip chart when the shape matters.

Ref.: Tukey, J. W. (1977). *Exploratory Data Analysis*. Addison-Wesley. — Original introduction of the box-and-whisker plot as an EDA tool.

Boxplot

Building on the density curves we just explored, the boxplot offers a compact, quartile-based summary that makes group comparisons immediately visible.

```
library(daltoolbox); library(ggplot2)
library(RColorBrewer)

colors <- brewer.pal(4, 'Set1')
font <- theme(text = element_text(size=16))

# Simple boxplot (white)
grf <- plot_boxplot(iris, colors="white") + font
# Note: uses Species as grouping variable
plot(grf)

# Colored boxplot by group
grf <- plot_boxplot(iris, colors=colors[1:4]) + font
plot(grf)
```

Step by step

- Pass the full iris dataset — `plot_boxplot()` automatically detects `Species` as the grouping variable
- Use `colors="white"` for a monochrome version
- Use `colors[1:4]` to color each group distinctly
- The "whiskers" indicate variability outside the quartiles; points beyond them are outliers

❏ **Key concept:** The boxplot summarizes the distribution via *quartiles* (Q1, median, Q3) and identifies outliers through the whiskers. It allows direct visual comparison between groups.

Finally, we conclude with our acknowledgements and references that made this toolkit possible.

Relationships Between Variables

A visual guide to **exploring relationships between two or more variables**: scatter plots and dot charts that reveal correlations, patterns, and clusters in data. Each visualization serves as a tool to communicate associations and trends between numerical attributes.

We'll begin by examining multi-dimensional comparisons with radar charts, then progressively explore distributions and statistical summaries. *Now let's explore each chart type in detail.*

Scatter Plot – Theory

A scatter plot maps two numeric variables onto Cartesian axes — one observation per point. It is the primary tool for detecting correlations, clusters, and outliers between pairs of variables, and is an essential first step before any regression or forecasting model.

Reading the pattern

A linear cloud of points suggests a linear relationship. A curved pattern suggests non-linearity. A circular cloud suggests no relationship. The tightness of the cloud reflects the strength of the association.

Adding a third dimension

Color, shape, or size of points can encode a categorical or numeric third variable (e.g., species, group), turning a 2D plot into a multivariate visualization.

Correlation vs. causation

A strong correlation in a scatter plot does not imply causation. Always interpret patterns in the context of domain knowledge and consider confounding variables.

❏ **"Before fitting any model, always plot your data. Anscombe's Quartet (1973) famously showed that four datasets with identical summary statistics can have completely different scatter plot patterns — a powerful reminder that visualization is irreplaceable."**

Ref.: Cleveland, W. S. (1993). *Visualizing Data*. Hobart Press. — A foundational reference on the principles of effective data visualization, including scatter plots and loess smoothing.

Scatter Plot

Use scatter plots to reveal **correlations and groupings** between two numeric variables — an essential first step before modeling trends or forecasting in time series analysis.

```
library(daltoolbox); library(ggplot2)
library(RColorBrewer); library(dplyr)

colors <- brewer.pal(4, 'Set1')
font <- theme(text = element_text(size=16))

# Rename columns to the helper's standard:
# x      = independent variable (X)
# value  = dependent variable (Y)
# variable = group/color

grf <- plot_scatter(
  iris |> dplyr::select(x = Sepal.Length,
    value = Sepal.Width, variable = Species),
  label_x = "Sepal.Length", label_y = "Sepal.Width",
  colors = colors[1:3]) + font

plot(grf)
```

Step by step

- **Rename columns** inside `select()` — map your X variable to `x`, your Y variable to `value`, and your grouping factor to `variable`
- **Define groups by color** — the `variable` column automatically assigns a distinct color to each group level
- **Supply one color per group** — pass a color vector to `colors` matching the number of factor levels
- **Label the axes** — use `label_x` and `label_y` to make the chart self-explanatory for your audience

❏ **Why it matters:** Before analyzing trends or building forecasts, it is critical to understand *how variables relate to each other*. The scatter plot is the simplest and most powerful tool for detecting correlations, outliers, and group structures — insights that directly shape every modeling decision that follows.

Points Chart – Theory

A points chart (or dot plot) displays individual observations as isolated points on a coordinate system — without connecting lines between them. It is the appropriate choice when observations are discrete and independent, and connecting them would falsely imply continuity or a sequential relationship.

Points vs. Lines

Lines imply that the path between two observations carries meaning (e.g., a trend over time). Points make no such claim — each observation stands alone. Use points when the X-axis is categorical or when observations are not ordered in a meaningful sequence.

Detecting structure

Even without lines, clusters, gaps, and outliers are clearly visible. Points charts are excellent for revealing the density and spread of observations across a range.

When to prefer points

Discrete events, cross-sectional data, or any situation where interpolation between observations is not meaningful. Also useful for overlaying raw data on top of summary statistics (e.g., mean $\pm$ SD).

❏ **"The choice between points and lines is a statement about the nature of your data. Lines encode continuity; points encode discreteness. Choosing the wrong one misleads the reader about the underlying data structure."**

Ref.: Wilkinson, L. (2005). *The Grammar of Graphics* (2nd ed.). Springer. — Discusses the grammar of geoms (`geom_point` vs. `geom_line`) and when each is semantically appropriate.

Points Chart (*without connecting lines*)

```
library(daltoolbox); library(ggplot2)
library(RColorBrewer)

colors <- brewer.pal(4, 'Set1')
font <- theme(text = element_text(size=16))

# Synthetic time series
x <- seq(0, 10, 0.25)
serie <- data.frame(x = x, sin = sin(x), cosine = cos(x) + 5)
# Points chart (no connecting lines)
grf <- plot_points(serie, colors=colors[1:2])
grf <- grf + font
plot(grf)
```

Step by step

- Build a `data.frame` with an `x` column and one column per series — here, `sin` and `cosine`
- Call `plot_points()` with the data and one color per series in `colors`
- Chain `+ font` to scale up the text size for readability
- Unlike `plot_series()`, **no lines connect the dots** — each observation stands alone

📌 **When to use points vs. lines:** Choose `plot_points()` when observations are *discrete and independent* — connecting them would imply a continuity that doesn't exist. Use `plot_series()` when the data represents a genuine sequence where the path between points carries meaning.

Time Series

A visual guide to understanding temporal data — organized by **analytical question**. From uncovering hidden patterns to projecting what comes next, each visualization is crafted to communicate **trends, structures, and forecasts** with clarity and precision.

Whether you're exploring a basic time series or building predictive models, this guide equips you with the right chart for every story your data wants to tell.

Time Series Visualization – Theory

A time series is a sequence of observations indexed in chronological order. Visualizing a time series with connected line segments is the most natural way to reveal its temporal structure — trends, seasonality, cycles, and irregular fluctuations — before any formal modeling.

Trend

A long-term increase or decrease in the level of the series. Visible as an upward or downward slope in the line plot.

Seasonality

A regular, repeating pattern tied to a fixed calendar period (e.g., daily, weekly, annual). Appears as periodic peaks and troughs at consistent intervals.

Cycles

Longer-term oscillations not tied to a fixed period (e.g., economic cycles). Harder to detect visually; require longer series.

Irregular component

Random noise remaining after trend and seasonality are accounted for. A very noisy series may benefit from smoothing (e.g., moving average) before visual inspection.

❏ **"Always plot your time series before modeling. Visual inspection reveals structural breaks, missing values, outliers, and non-stationarity — all of which must be addressed before applying forecasting methods."**

Ref.: Hyndman, R. J. & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. Available at: <https://otexts.com/fpp3/> — Open-access textbook covering time series visualization, decomposition, and forecasting.

Time Series with Lines

```
library(daltoolbox); library(ggplot2)
library(RColorBrewer)

colors <- brewer.pal(4, 'Set1')
font <- theme(text = element_text(size=16))

# Série sintética: seno e cosseno deslocado
x <- seq(0, 10, 0.25)
serie <- data.frame(x = x, sin = sin(x), cosine = cos(x) + 5)
# Gráfico de séries (pontos + linhas)
grf <- plot_series(serie, colors=colors[1:2]) + font
plot(grf)
```

Step by step

- Generate the ordered X axis with `seq()`
- Build the `data.frame` with `x` and the series as columns
- Call `plot_series()` — connects points with line segments
- Pass `colors[1:2]` to distinguish the two series

📄 **Key concept:** The series plot connects points with line segments with the X axis ordered by time/index. It highlights *trend and seasonality* over time.

Forecasting Visualization – Theory

Visualizing a forecast means displaying three distinct components together: the observed historical values, the model's fitted values (in-sample), and the predicted future values (out-of-sample). A well-designed forecast plot makes the quality of the fit and the uncertainty of the prediction immediately apparent.

Observed vs. Fitted

The fitted line shows how well the model captures the historical pattern. Large deviations between observed and fitted values indicate poor in-sample fit and suggest the model may not generalize well.

Forecast horizon

The predicted values extend beyond the last observed point. The length of the horizon should be chosen carefully: short-term forecasts are generally more reliable than long-term ones, as uncertainty compounds over time.

Evaluating the forecast visually

A good forecast should: (a) track the observed series closely in the fit window, (b) continue plausible patterns into the forecast window, and (c) show uncertainty bounds (confidence intervals) that widen with the horizon.

❏ **"Visual evaluation of a forecast is not a substitute for quantitative metrics (MAE, RMSE, MAPE), but it is an essential complement. Plots reveal systematic biases and structural failures that aggregate metrics can hide."**

Ref.: Hyndman, R. J. & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/> — Chapters on forecast evaluation and visualization of prediction intervals.

Simple Time Series and with Forecast

```
library(daltoolbox)
library(ggplot2)
library(RColorBrewer)

colors <- brewer.pal(4, 'Set1')
font <- theme(text = element_text(size=16))

# --- Simple series (plot_ts) ---
x <- seq(0, 10, 0.25)
y <- sin(x)

grf <- plot_ts(x=x, y=y, color=c("red"))
plot(grf)

# --- Series with fit and forecast ---
x <- base::seq(0, 10, 0.25)
yvalues <- sin(x) + rnorm(41, 0, 0.1) # observed with noise

adjust <- sin(x[1:35]) # fit (train)
prediction <- sin(x[36:41]) # forecast (test)

plot_ts_pred(y = yvalues, yadj = adjust, ypre = prediction)
```

Step by step – Simple Series

- `plot_ts(x, y, color)`: basic exploratory visualization of a time vector
- Ordered X axis; Y axis with series values

Step by step – With Forecast

- Add noise with `rnorm()` to simulate real data
- Separate windows for **fit** (`x[1:35]`) and **forecast** (`x[36:41]`)
- `plot_ts_pred()` displays all three elements: observed, fit, and forecast

📌 **Key concept:** `plot_ts_pred()` combines *observed values*, *model fit*, and *forecast horizon* in a single chart — essential for visual evaluation of time series models.

Exporting Charts

Learn how to **save charts in different formats** — PDF, JPEG, and PNG — using the `ggsave` function. This section presents how to export your visualizations with precise control over dimensions, resolution, and quality, ensuring results ready for publication or presentation.

Chart Export Formats – Theory

Exporting a chart correctly is as important as creating it. The choice of file format, resolution, and dimensions directly affects how the visualization will appear in its final destination — whether a scientific paper, a presentation, a website, or a report.

Vector formats (PDF, SVG)

Resolution-independent: the chart can be scaled to any size without loss of quality. Ideal for publications, LaTeX documents, and any context where the chart may be resized. PDF is the standard for academic papers; SVG is preferred for web use.

Raster formats (PNG, JPEG)

Resolution-dependent: quality is fixed at export time (measured in DPI — dots per inch). PNG is lossless and preferred for charts with sharp edges and text. JPEG uses lossy compression and is better suited for photographs; avoid it for charts with fine lines or text.

Resolution guidelines

Screen display: 72–96 DPI. Presentations: 150 DPI. Print/publication: 300 DPI minimum. Always specify dimensions explicitly (width × height in cm or inches) to ensure consistent rendering across environments.

❏ **"A common mistake is exporting at screen resolution (72 DPI) for print use — the result looks blurry. Always set DPI explicitly with `ggsave(dpi = 300)` for publication-quality output."**

Ref.: Wickham, H. (2016). *ggplot2: Elegant Graphics for Data Analysis* (2nd ed.). Springer. — Chapter on saving plots covers `ggsave()` options, device types, and resolution settings.

Saving Charts: PDF, JPEG, and PNG

```
library(daltoolbox); library(ggplot2)
library(RColorBrewer)

colors <- brewer.pal(4, 'Set1')
font <- theme(text = element_text(size=16))

# Chart to export
x <- seq(0, 10, 0.25)
serie <- data.frame(x, sin=sin(x), cosine=cos(x)+5)
grf <- plot_series(serie, colors=colors[1:2]) + font
```

```
# --- Export to PDF with ggsave ---
ggsave("series.pdf", width=5, height=4, units="cm")

# --- Export to PNG with ggsave ---
ggsave("series.png", width=5, height=4, units="cm")

# --- Export to JPEG with base device ---
jpeg("series.jpg", width=640, height=480)
plot(grf)
dev.off()
```

Step by step

- Create and plot the chart normally before exporting
- `ggsave()`: saves the last ggplot2 chart; control dimensions with `width`, `height`, and `units`
- For JPEG via base device: open with `jpeg()`, plot with `plot(grf)`, and close with `dev.off()`
- `ggsave()` detects the format from the filename (`.pdf`, `.png`, `.jpg`)

❏ **Key concept:** `ggsave()` is the recommended way to export ggplot2 charts. The base device (`jpeg()` + `dev.off()`) is an alternative for raster formats with pixel-level resolution control.

Acknowledgements



Data Analytics Lab

dal.global@grupo.cefet-rj.br | <https://eic.cefet-rj.br/dal>

